

디지털 포렌식 **CTF** - 은닉 아티팩트 분석 프로젝트

프로젝트 유형: 디지털 포렌식 CTF 분석

분석 환경: Kali Linux

사용 도구: strings, grep, hexdump, dd, file, xdg-open, hexedit, exiftool, jadx, base64

분석 분야: 은닉 데이터 탐지, 파일 구조 분석, 파일 카빙, 메타데이터 분석, **APK** 정적 분석

분석 결과: 다양한 파일 유형(텍스트, 실행 파일, 이미지, **APK**)에 적용된 은닉 기법을 분석하고, 파일 구조 기반 포렌식 절차를 통해 복수의 숨겨진 아티팩트를 식별함

작성일: 2025.6

버전: v1.0

GitHub: <https://github.com/Minseo147/cybersecurity-portfolio>

Blog: <https://cyberlab.tistory.com/>

목차

1. 프로젝트 개요.....	3
2. 분석 목적.....	3
3. 증거 관리 및 무결성 유지.....	4
3.1 증거 수집.....	4
3.2 무결성 관리 인식.....	4
3.3 분석 환경 분리.....	5
4. 분석 환경 구성.....	5
5. 분석 방법론.....	6
5.1 1단계: 초기 분류.....	6
5.2 2단계: 구조 분석.....	6
5.3 3단계: 내용 분석.....	6
5.4 4단계: 심층 바이너리 분석 및 카빙.....	7
5.5 5단계: 결과 검증 및 문서화.....	7
6. 분석 결과.....	8
6.1 문제 1: 숨겨진 텍스트 추출.....	8
6.2 문제 2: 실행 파일 내 PNG 삽입 데이터 추출.....	9
6.3 문제 3: APK 정적 분석.....	10
6.4 문제 4: 파일 헤더 및 메타데이터 분석.....	12
6.5 문제 5: 바이너리 문자열 기반 은닉 데이터 탐지.....	13
7. 적용 기술.....	14
8. 실무 적용 가능성.....	15
9. 문제 해결 과정 및 학습 내용.....	15
10. 프로젝트 회고.....	16

1. 프로젝트 개요

본 프로젝트는 서로 다른 파일 유형이 혼재된 디렉토리 환경에서 은닉 데이터 존재 가능성을 가정하고 수행한 디지털 포렌식 조사이다. 분석 대상에는 텍스트 파일, 실행 파일, 이미지 파일, 모바일 애플리케이션 패키지(APK) 등 구조와 포맷이 서로 다른 파일이 포함되어 있었으며, 각 파일은 정상적인 외형을 유지하면서도 내부적으로는 숨겨진 데이터를 포함하고 있을 가능성이 있었다. 이에 따라 본 조사는 단순한 문제 풀이가 아니라, 파일 구조와 데이터 표현 방식을 고려한 단계적 포렌식 분석 절차를 기반으로 수행되었다.

조사 과정에서는 문자열 기반 탐지, 파일 시그니처 검증, 바이너리 오프셋 분석, 파일 카빙, 메타데이터 분석, 인코딩 데이터 복호화, APK 정적 분석을 순차적으로 적용하였다. 특히 각 파일에 동일한 방식으로 접근하지 않고, 파일 유형에 따라 가장 가능성이 높은 은닉 방식을 먼저 가정한 뒤 분석 절차를 조정하였다. 이러한 접근은 단순히 플래그를 찾는 데 그치지 않고, 왜 해당 파일이 의심스러웠는지와 왜 특정 도구를 적용했는지를 설명 가능한 조사 흐름으로 정리했다는 점에서 의미가 있다.

분석 결과, 총 다섯 가지 서로 다른 은닉 방식을 식별하였다. 첫 번째는 텍스트 파일 내부에 직접 삽입된 평문 문자열이었고, 두 번째는 실행 파일 내부에 삽입된 PNG 이미지였다. 세 번째는 APK 리소스 내부에 저장된 Base64 인코딩 문자열이었으며, 네 번째는 손상된 JPEG 파일을 복구한 뒤 XMP 메타데이터에서 확인한 은닉 정보였다. 마지막은 시각적으로는 정상적인 이미지처럼 보이지만, 바이너리 내부 문자열 추출을 통해 확인된 평문 데이터였다.

본 프로젝트의 핵심 의미는 단순히 여러 플래그를 찾았다는 데 있지 않다. 더 중요한 점은, 각 아티팩트가 어떤 구조적 특성을 가지고 있었는지, 왜 해당 파일을 의심했는지, 어떤 절차를 통해 숨겨진 정보를 확인했는지를 재현 가능한 형태로 문서화했다는 점이다. 이러한 경험은 실제 침해 사고 대응, 악성코드 정적 분석, 엔드포인트 포렌식, 디지털 증거 분석에서 요구되는 기본 역량과 직접적으로 연결된다.

2. 분석 목적

본 분석은 다수의 파일이 포함된 디렉토리 환경에서 비정상적 데이터 은닉 가능성이 존재한다는 가정 하에 수행되었다. 분석 대상에는 실행 파일, 이미지 파일, 텍스트 파일, APK 등 다양한 파일 유형이 혼재되어 있었으며, 일부 파일은 겉으로는 정상처럼 보이지만 내부 구조가 변조되었거나 별도 데이터를 포함하고 있을 가능성이 있었다. 실제 디지털 포렌식 조사에서도 공격자는 파일 확장자나 외형은 정상으로 유지한 채, 내부에 악성 데이터나 숨겨진 정보를 삽입하는 방식으로 탐지를 우회하려 한다. 본 프로젝트는 이러한 상황을 가정하고, 파일 구조 및 내부 데이터 분석을 통해 은닉 정보를 식별하는 절차를 직접 적용하는 데 목적이 있었다.

조사의 주요 목표는 다음과 같다.

- 파일 내부에 은닉된 비인가 데이터 존재 여부 확인
- 파일 시그니처 및 헤더 무결성 검증

- 바이너리 구조 분석을 통한 숨겨진 데이터 블록 탐지
- 인코딩 또는 변조된 데이터 복원
- 분석 절차의 재현 가능성 확보

본 프로젝트에서는 다음과 같은 가설을 설정하였다.

“파일 내부에는 정상 파일 구조를 가정한 은닉 데이터가 존재할 가능성이 있다.”

이에 따라 단순 문자열 검색만으로 결론을 내리기보다, 파일 구조 분석 → 내부 데이터 확인 → 은닉 가능성 평가 → 결과 검증의 흐름으로 조사 절차를 설계하였다. 이는 CTF 환경에 국한되지 않고, 실제 파일 기반 포렌식 조사에서도 적용 가능한 기본 프레임워크이다.

3. 증거 관리 및 무결성 유지

3.1 증거 수집

본 분석은 제공된 압축 파일을 원본 증거로 간주하고 시작하였다. 학습용 데이터셋이지만, 디지털 포렌식의 기본 원칙은 교육 환경에서도 동일하게 적용되어야 한다고 판단하였다. 따라서 분석 초기 단계에서는 원본 데이터의 무결성 보존을 우선적으로 고려하였다.

우선 원본 압축 파일은 별도의 디렉토리에 보관하고, 실제 분석은 압축 해제 후 생성한 작업용 복사본에서만 수행하였다. 이는 분석 도중 파일을 수정하거나 손상시키는 상황을 방지하기 위한 조치이다. 특히 파일 헤더를 복구해야 하는 **Challenge 4**와 같이 원본 상태를 직접 변경하게 되는 경우, 작업용 복사본을 사용하는 것은 필수적이다. 분석 결과물 역시 원본과 분리된 별도 디렉토리에 저장하여, 원본 상태와 생성 산출물을 명확히 구분할 수 있도록 하였다.

이러한 절차는 실제 포렌식 환경에서 요구되는 증거 보존 원칙을 반영한 것이다. 분석가는 증거를 이해하고 해석하는 역할을 수행해야 하며, 원본 증거 자체를 변경해서는 안 된다. 본 프로젝트는 학습 환경에서 수행되었지만, 실제 조사 관점에서 원본 보존과 작업 환경 분리를 전제로 접근했다는 점에서 의미가 있다.

3.2 무결성 관리 인식

본 프로젝트에서는 공식적인 해시값 제출이 요구되지는 않았지만, 디지털 포렌식에서 무결성 검증이 왜 중요한지에 대한 인식을 바탕으로 작업을 진행하였다. 실무에서 해시값은 단순 체크값이 아니라, 분석 대상 파일이 원본과 동일한 상태였음을 입증하는 핵심 수단이다. 특히 법적 제출이나 내부 감사 상황에서는 “해당 파일이 분석 시점까지 변조되지 않았는가”를 설명할 수 있어야 한다.

일반적으로 포렌식 조사는 다음과 같은 무결성 관리 절차를 포함한다.

- MD5 또는 SHA256 해시 생성
- 해시값 기록 및 보고서 명시
- 분석 전후 해시 비교
- Chain of Custody 문서화

본 프로젝트에서는 교육용 데이터셋이라는 한계상 공식 해시 산출과 제출까지는 수행하지 않았으나, 적어도 다음 원칙을 유지하였다. 첫째, 원본은 직접 수정하지 않는다. 둘째, 수정이 필요한 경우 작업용 복사본에서 수행한다. 셋째, 분석 결과물은 원본과 분리해서 저장한다. 넷째, 어떤 도구를 어떤 이유로 사용했는지 문서화한다. 이는 해시 계산 자체보다도, 무결성 관리의 필요성과 그 절차를 이해하고 있다는 점을 보여주는 요소라고 판단하였다.

3.3 분석 환경 분리

분석은 **Kali Linux CLI** 환경에서 수행하였다. 작업 환경을 분리한 이유는 증거 오염 방지와 조사 재현성 확보 때문이다. 포렌식 분석은 일반 사용자 환경보다 통제된 분석 환경에서 이루어져야 하며, 원본 파일과 작업 산출물이 뒤섞이지 않도록 관리되어야 한다.

본 프로젝트에서는 원본 디렉토리와 작업 디렉토리를 분리하였고, 카빙된 **PNG** 파일, 복구된 **JPEG** 파일, **APK** 디컴파일 결과물 등 모든 산출물을 별도의 경로에 저장하였다. 이렇게 함으로써 어떤 파일이 원본인지, 어떤 파일이 분석 과정에서 생성된 것인지 명확하게 구분할 수 있었다. 또한 **CLI** 환경을 사용함으로써 각 분석 단계의 명령어 자체가 절차 기록이 되도록 하였다. 이는 동일한 환경에서 동일한 분석을 반복 수행하기 용이하다는 점에서 포렌식 실무와 포트폴리오 문서화 양쪽 모두에 장점이 있다.

4. 분석 환경 구성

본 분석은 **Kali Linux** 환경에서 수행하였다. **Kali Linux**는 포렌식 및 보안 분석에 자주 사용되는 도구를 폭넓게 제공하며, **CLI** 기반 반복 분석과 검증에 적합하다. 특히 문자열 추출, 시그니처 확인, 카빙, 메타데이터 분석, **APK** 정적 분석 등 서로 다른 유형의 작업을 하나의 환경에서 일관되게 수행할 수 있다는 점에서 본 프로젝트와 잘 맞았다.

본 프로젝트에서 사용한 주요 도구는 다음과 같다.

- **strings**: 파일 내부의 **ASCII** 문자열 추출
- **grep**: 특정 패턴 필터링
- **hexdump**: 바이너리 hex 구조 확인
- **dd**: 오프셋 기반 파일 카빙
- **file**: 파일 포맷 식별
- **xdg-open**: 복구·추출 파일의 시각적 검증
- **hexedit**: 손상된 헤더 수정
- **exiftool**: 메타데이터 분석
- **jadx**: **APK** 디컴파일 및 정적 분석
- **base64**: 인코딩 문자열 디코딩

도구 선택 기준은 단순히 많이 알려진 도구를 나열하는 데 있지 않았다. 각 단계에서 “왜 이 도구가 필요한가”를 우선 고려하였다. 예를 들어 실행 파일 내부 삽입 데이터를 분석할 때 바로 고급 카빙 도구를 사용하는 대신, 먼저 **hexdump**를 통해 시그니처 위치를 확인한 후

dd로 수동 추출을 수행하였다. 이는 결과만 얻는 것보다 파일 구조를 이해하고, 추출 논리를 설명할 수 있는 방식에 더 가깝다고 판단했기 때문이다.

5. 분석 방법론

5.1 1단계: 초기 분류

초기 분류 단계에서는 전체 디렉토리 구조와 파일 유형을 확인하였다. 이 단계의 목적은 즉시 세부 분석에 들어가는 것이 아니라, 무엇이 조사 대상인지 파악하고 어떤 파일이 우선적으로 분석 가치가 높은지 판단하는 데 있다.

수행 내용은 다음과 같다.

- 디렉토리 구조 확인
- 파일명 및 확장자 확인
- 텍스트, 실행 파일, 이미지, APK 등 파일 유형 분류
- 이상 파일 여부 1차 판단

이 단계는 Incident Response의 초기 triage와 유사하다. 모든 파일을 동일한 깊이로 먼저 분석하는 것은 비효율적이기 때문에, 파일 성격에 따라 가능한 은닉 방식을 먼저 가정하는 것이 필요하다. 본 프로젝트에서는 텍스트 파일은 평문 검색, 실행 파일은 삽입 파일 가능성, 이미지 파일은 헤더 변조나 메타데이터, APK는 리소스·문자열 분석 가능성을 중심으로 1차 가설을 세웠다.

5.2 2단계: 구조 분석

구조 분석 단계에서는 파일 포맷 무결성과 내부 구조를 검증하였다. 확장자만으로 파일 형식을 판단하는 것은 신뢰성이 낮기 때문에, 실제 시그니처와 헤더 값을 기준으로 파일 구조가 정상인지, 혹은 고의로 변조되었는지 확인하였다.

수행 내용은 다음과 같다.

- 매직 넘버 확인
- 정상 헤더 여부 검증
- 포맷 위조 가능성 평가
- 손상 파일 복구 가능성 검토

이 단계는 Challenge 2와 Challenge 4에서 특히 중요했다. 실행 파일 내부 PNG 삽입은 시그니처 확인 없이는 발견하기 어려웠고, JPEG 헤더 손상 역시 구조를 확인하지 않았다면 단순 오류 파일로 오인했을 가능성이 있다. 따라서 구조 분석은 “보이는 대로 믿지 않는 포렌식적 사고”의 핵심 단계였다.

5.3 3단계: 내용 분석

내용 기반 분석 단계에서는 문자열, 인코딩 데이터, 메타데이터, 바이너리 내부 평문 데이터를 확인하였다. 구조가 정상이라고 해도 내부 내용까지 정상이라는 보장은 없기 때문에, 사람이 읽을 수 있는 정보가 어디에든 남아 있는지 확인하는 과정이 필요하다.

수행 내용은 다음과 같다.

- ASCII 문자열 추출
- 특정 패턴 기반 검색
- Base64 인코딩 문자열 식별 및 디코딩
- EXIF/XMP 메타데이터 분석
- 이미지 바이너리 내부 문자열 확인

실무에서도 악성코드, 앱, 문서, 이미지 파일 내부에는 계정 정보, URL, 명령 문자열, 토큰, 디버그 흔적이 평문 또는 단순 인코딩 상태로 남아 있는 경우가 있다. 따라서 strings나 exiftool과 같은 기본 도구는 여전히 강력한 1차 분석 도구이다.

5.4 4단계: 심층 바이너리 분석 및 카빙

심층 바이너리 분석 단계에서는 단순 문자열 검색으로 해결되지 않는 파일을 대상으로 hex 구조 확인과 파일 카빙을 수행하였다. 이 단계는 파일 내부 특정 오프셋부터 다른 포맷이 시작되거나, 실행 파일 내부에 별도의 파일이 삽입된 상황에서 특히 유효하다.

수행 내용은 다음과 같다.

- Hex 단위 분석
- 시그니처 기반 오프셋 탐색
- dd를 이용한 수동 카빙
- 추출 파일 포맷 검증

Challenge 2의 경우, 평문 검색 실패 후 구조 분석으로 방향을 전환했기 때문에 최종적으로 PNG 이미지를 추출할 수 있었다. 이처럼 카빙은 단순 추출 기술이 아니라, 왜 특정 위치부터 데이터를 잘라야 하는지를 설명할 수 있어야 의미가 있다는 점을 확인할 수 있었다.

5.5 5단계: 결과 검증 및 문서화

마지막 단계는 결과 검증과 문서화이다. 포렌식 분석은 결과를 얻는 것에서 끝나지 않고, 해당 결과가 타당하며 다른 분석가가 같은 절차로 재현 가능해야 한다. 따라서 본 프로젝트에서는 각 결과에 대해 가능한 범위 내에서 교차 검증을 수행하였다.

예를 들어, 카빙한 PNG 파일은 file 명령어로 포맷을 다시 확인하고, xdg-open으로 시각적 검증을 하였다. JPEG 파일은 헤더 복구 후 정상 열람 여부를 확인한 뒤 exiftool로 메타데이터를 분석하였다. APK는 디컴파일 후 리소스 파일을 직접 열어 문자열을 확인하고, Base64 디코딩 결과와 일치하는지 검토하였다.

문서화 역시 단순 명령어 나열에 그치지 않고, 왜 그 명령어를 사용했는지, 어떤 근거로 다음 분석 단계로 넘어갔는지, 결과가 무엇을 의미하는지까지 포함하도록 구성하였다. 이것이 단순 과제 풀이와 포트폴리오 보고서를 구분하는 핵심 요소라고 판단하였다.

6. 분석 결과

본 프로젝트는 하나의 교육용 디지털 증거 세트를 대상으로 수행된 구조적 포렌식 조사이며, 각 분석 항목은 앞서 정의한 단계적 분석 프레임워크에 따라 수행되었다. 각 **Challenge**는 서로 다른 유형의 은닉 방식을 포함하고 있었기 때문에, 분석 과정에서 관찰 결과에 따라 다음 가설을 조정하는 방식으로 접근하였다.

6.1 문제 1: 숨겨진 텍스트 추출

분석 목적

텍스트 파일 내부에 숨겨진 평문 데이터를 식별한다.

사용 도구

ls	cd	grep
----	----	------

분석 과정

초기 분류 단계에서 문제 1에는 텍스트 파일이 존재함을 확인하였다. 텍스트 파일은 구조가 단순하고 평문 기반 정보가 직접 포함될 가능성이 높기 때문에, 우선적으로 패턴 검색을 수행하는 것이 합리적이라고 판단하였다. 실제 포렌식 조사에서도 설정 파일, 로그 파일, 메모 파일 등은 다른 복잡한 분석보다 먼저 평문 검색을 수행하는 경우가 많다.

디렉토리 이동 후 파일명을 확인하고, 플래그 패턴을 기준으로 문자열 검색을 수행하였다.

```
grep ANS The*****Physics.txt
```

분석 결과

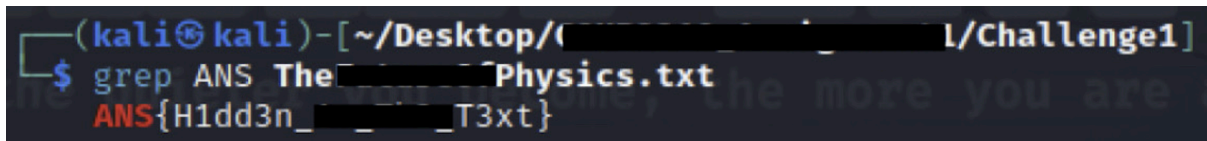
텍스트 파일 내부에서 평문 플래그가 직접 확인되었다.

```
Flag: ANS{H1dd3n_****_T3xt}
```

분석 해석

이 문제는 가장 단순한 유형이지만, 초기 **triage**의 중요성을 잘 보여준다. 모든 분석이 복잡한 도구를 필요로 하는 것은 아니며, 기본적인 패턴 검색만으로도 유의미한 증거를

빠르게 확보할 수 있다. 본 사례는 ASCII 기반 평문 데이터 탐지 능력과, 가장 간단한 경로부터 검토하는 조사 습관을 보여준다.



```
(kali㉿kali)-[~/Desktop/(...)/Challenge1]
$ grep ANS The...Physics.txt
ANS{H1dd3n_...T3xt}
```

Figure 1. grep 명령어를 사용하여 텍스트 파일 내부에서 키워드 기반으로 평문 플래그를 식별한 결과

6.2 문제 2: 실행 파일 내 PNG 삽입 데이터 추출

분석 목적

실행 파일 내부에 삽입된 이미지 데이터를 탐지하고 추출한다.

사용 도구

ls -l	strings	hexdump	dd	file	xdg-open
-------	---------	---------	----	------	----------

분석 과정

문제 2에서는 *****.exe 파일이 제공되었다. 파일명과 확장자를 고려했을 때, 초기 단계에서는 내부 문자열에 직접적인 힌트가 남아 있을 가능성을 검토하였다. 이에 따라 다음 명령어로 평문 문자열을 확인하였다.

```
strings *****.exe | grep ANS
```

그러나 이 단계에서는 플래그가 발견되지 않았다. 이 결과는 단순히 “정보가 없다”라기보다, 평문 대신 다른 포맷의 데이터가 삽입되었을 가능성을 시사하였다. 실행 파일 내부에 이미지나 추가 블록이 삽입되는 구조는 실제 악성코드에서도 자주 관찰되므로, 다음 단계로 바이너리 시그니처 확인을 수행하였다.

```
hexdump -C *****.exe | grep '89 ** * 47'
```

분석 결과 PNG 시그니처가 확인되었다. 이는 실행 파일 내부에 PNG 데이터가 포함되어 있음을 의미한다. 이후 dd를 사용해 데이터를 수동 추출하였다.

```
dd if=*****.exe of=extracted_flag_image.png bs=1 skip=0 count=8252
```

추출 후에는 file 명령어로 포맷을 검증하고, xdg-open으로 실제 이미지 내용을 확인하였다.

분석 결과

카빙된 PNG 이미지 내부에서 플래그가 시각적으로 확인되었다.

Flag: ANS{Nice And ***** To Copy}

분석 해석

이 문제는 문자열 검색 실패 이후 분석 방향을 적절히 바꾼 사례이다. 실행 파일 내부 삽입 데이터는 실무에서도 흔한 은닉 방식이며, 파일 시그니처 기반 접근과 수동 카빙을 통해 구조적 증거를 직접 추출할 수 있어야 한다. 본 사례는 단순 도구 사용을 넘어, “왜 내부 삽입 파일을 의심했는가”를 설명할 수 있다는 점에서 의미가 있다.

```
(kali㉿kali)-[~/Desktop/[REDACTED]/Challenge2]
$ strings [REDACTED].exe | grep ANS
```

Figure 2. 초기 문자열 검색(strings + grep) 과정에서 유의미한 평문 데이터가 확인되지 않은 결과

```
(kali㉿kali)-[~/Desktop/[REDACTED]/Challenge2]
$ hexdump -C [REDACTED].exe | grep '89 47'
00000000 [REDACTED] 0 0d 49 48 44 52 |.PNG.....IHDR
```

Figure 3. hexdump를 통해 실행 파일 내부에서 PNG 파일 시그니처를 탐지한 결과

```
(kali㉿kali)-[~/Desktop/[REDACTED]/Challenge2]
$ dd if=[REDACTED].exe of=extracted_flag_image.png bs=1 skip=0 count=8252
8252+0 records in
8252+0 records out
8252 bytes (8.3 kB, 8.1 KiB) copied, 0.0124679 s, 662 kB/s

(kali㉿kali)-[~/Desktop/[REDACTED]/Challenge2]
$ file extracted_flag_image.png
extracted_flag_image.png: PNG image data, 800 x 600, 8-bit colormap, non-interlaced
```

Figure 4. dd 명령어를 사용하여 실행 파일 내부에 삽입된 PNG 데이터를 추출하고 file 명령어로 검증한 과정

ANS{Nice And [REDACTED] To Copy}

Figure 5. 카빙된 PNG 이미지를 통해 시각적으로 플래그를 확인한 결과

6.3 문제 3: APK 정적 분석

분석 목적

APK 파일 내부의 은닉 데이터를 정적으로 분석하여 복원한다.

사용 도구

jadx	cat	base64
------	-----	--------

분석 과정

문제 3에서는 *****.apk 파일이 제공되었다. APK는 단순 파일이 아니라 리소스, XML, 클래스, 문자열, 설정 정보를 포함하는 구조화된 패키지이므로, 문자열 검색보다 정적 분석이 더 적합하다고 판단하였다. 우선 JADX를 사용하여 디컴파일을 수행하였다.

```
jadx -d jadx_output *****.apk
```

디컴파일된 리소스 구조를 확인한 결과, res/xml/ANS.txt 파일이 존재하였다. 파일명을 고려했을 때 조사 대상 가치가 높다고 판단하여 내용을 직접 확인하였다.

```
cat jadx_output/resources/res/xml/ANS.txt
```

해당 파일에는 Base64 형식으로 보이는 문자열이 포함되어 있었으며, 이는 평문 노출을 피하기 위한 단순 인코딩으로 보였다. 따라서 다음과 같이 디코딩을 수행하였다.

```
cat jadx_output/resources/res/xml/ANS.txt | base64 -d
```

분석 결과

Base64 디코딩 결과 최종 플래그가 복원되었다.

```
Flag: ANS{44423A089*****6EBD16666D467}
```

분석 해석

본 문제는 모바일 앱 정적 분석의 기본 흐름을 보여준다. 실제 모바일 애플리케이션이나 악성 앱에서도 문자열, 토큰, URL, 설정값이 단순 인코딩 상태로 저장되는 경우가 많다. 본 사례는 APK 내부 리소스 구조 탐색, 인코딩 데이터 식별, 문자열 복원 능력을 보여주는 분석이다.

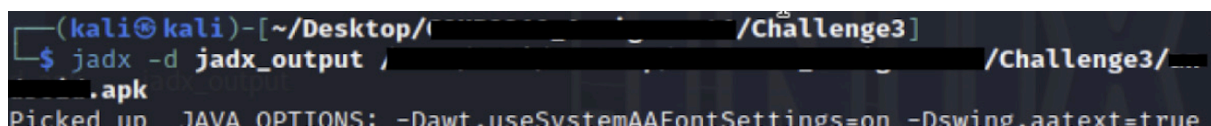


Figure 6. JADX 도구를 사용하여 APK 파일을 디컴파일하고 분석 환경을 구성한 과정

```
(kali㉿kali)-[~/Desktop/Challenge3]
$ ls jadx_output/resources/res/xml/
ANS.txt standalone_badge_gravity_bottom_end.xml
backup_rules.xml standalone_badge_gravity_bottom_start.xml
data_extraction_rules.xml standalone_badge_gravity_top_start.xml
standalone_badge.xml standalone_badge_offset.xml
```

Figure 7. 디컴파일된 APK 리소스 구조에서 ANS.txt 파일이 존재함을 확인한 결과

```
(kali㉿kali)-[~/Desktop/Challenge3]
$ cat jadx_output/resources/res/xml/ANS.txt
QU5TezQ
more you are able to hear"
```

Figure 8. APK 내부 ANS.txt 파일에서 인코딩된 문자열을 추출한 과정

```
(kali㉿kali)-[~/Desktop/Challenge3]
$ cat jadx_output/resources/res/xml/ANS.txt | base64 -d
ANS{44423A0895EBD16666D467}
```

Figure 9. Base64 디코딩을 통해 인코딩된 데이터를 복원하여 플래그를 확인한 결과

6.4 문제 4: 파일 헤더 및 메타데이터 분석

분석 목적

손상되거나 변조된 JPEG 파일을 복구하고, 메타데이터 기반 은닉 정보를 분석한다.

사용 도구

hexedit	exiftool	xdg-open
---------	----------	----------

분석 과정

문제 4에서는 ****.jpeg 파일이 정상적으로 열리지 않았다. 같은 디렉토리의 힌트 파일은 파일이 손상되었으며 수정이 필요하다는 점을 암시하고 있었다. 이에 따라 가장 먼저 파일 헤더 변조 가능성을 검토하였다.

JPEG 파일은 일반적으로 FF D8 FF E0 계열의 시작 시그니처를 가진다. hexedit로 파일 시작 부분을 확인한 결과, 정상적인 JPEG 시그니처 대신 DE AD가 포함되어 있었다. 이는 파일이 완전히 손상된 것이 아니라, 인식 방해를 위해 헤더 일부가 고의로 변조되었을 가능성을 강하게 시사했다. 따라서 시작 바이트를 정상 JPEG 헤더로 수정한 후 파일을 저장하였다.

이후 이미지를 다시 열어본 결과 시각적으로는 정상적으로 표시되었으나, 이미지 내부에 플래그가 직접 보이지는 않았다. 이에 따라 다음 분석 대상으로 메타데이터 영역을 고려하였다. exiftool을 사용해 EXIF 및 XMP 필드를 확인한 결과, Author 항목에서 플래그를 확인할 수 있었다.

```
exiftool ****.jpeg
```

분석 결과

복구된 JPEG 파일의 XMP Author 메타데이터에서 플래그가 확인되었다.

```
Flag: ANS{Y0u_*****_S3cr3t}
```

분석 해석

이 문제는 파일이 열리지 않는다고 해서 곧바로 무가치한 파일로 판단해서는 안 된다는 점을 보여준다. 또한 시각적으로는 정상인 파일이라 하더라도 메타데이터 영역에는 별도의 은닉 정보가 존재할 수 있다. 실제 포렌식 실무에서도 EXIF, XMP, IPTC 등의 메타데이터는 위치 정보, 작성자 정보, 편집 이력, 주석, 숨겨진 코멘트 등 중요한 증거를 포함할 수 있다. 본 분석은 파일 헤더 구조 이해, 손상 파일 복구, 메타데이터 기반 정보 탐지 역량을 보여준다.

```
(kali@kali)-[~/Desktop/Challenge4]
$ exiftool ****.jpeg
ExifTool Version Number      : 13.10
File Name                    : ****.jpeg
Directory                   : .
File Size                    : 9.0 kB
File Modification Date/Time  : 2025:04:02 03:42:27-07:00
File Access Date/Time       : 2025:04:02 03:42:39-07:00
File Inode Change Date/Time  : 2025:04:02 03:42:27-07:00
File Permissions             : -rwxr-xr-x
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                 : 1.01
Resolution Unit              : None
X Resolution                  : 1
Y Resolution                  : 1
XMP Toolkit                  : Image::ExifTool 13.10
Author                       : ANS{Y0u_*****_S3cr3t}
Image Width                  : 225
Image Height                  : 225
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling         : YCbCr4:2:0 (2 2)
Image Size                   : 225x225
Megapixels                   : 0.051
```

Figure 10. exiftool을 사용하여 JPEG 파일의 메타데이터를 분석하고 Author 필드에서 플래그를 확인한 결과

6.5 문제 5: 바이너리 문자열 기반 은닉 데이터 탐지

분석 목적

정상적으로 보이는 PNG 파일 내부에서 평문 기반 은닉 문자열을 탐지한다.

사용 도구

strings	grep	xdg-open
---------	------	----------

분석 과정

문제 5에서는 `*****.png` 파일이 제공되었다. 시각적으로 이미지를 확인했을 때는 단순한 자연 이미지로 보였으며, 플래그는 화면상에서 드러나지 않았다. 그러나 PNG 파일은 내부 바이너리 구조상 추가 문자열이나 불필요한 데이터를 포함하고 있어도 정상적으로 열릴 수 있기 때문에, 시각적 결과만으로 판단하지 않았다.

이에 따라 기본적인 문자열 추출을 수행하였다.

```
strings *****.png | grep ANS
```

그 결과 바이너리 내부에서 평문 플래그 문자열이 직접 확인되었다.

분석 결과

PNG 이미지 바이너리 내부에서 평문 문자열 형태의 플래그가 확인되었다.

```
Flag: ANS{Th1s_*****_c0de}
```

분석 해석

이 방식은 고급 스테가노그래피보다는 파일 내부에 사람이 읽을 수 있는 문자열을 덧붙이거나 포함시키는 단순 은닉 방식에 가깝다. 그럼에도 불구하고 일반 사용자는 이미지만 보고 해당 정보를 알기 어렵기 때문에 최소한의 은닉 효과는 있다. 실제 악성코드 분석에서도 이미지·문서·바이너리 내부에 URL, 계정 정보, 토큰, 명령 문자열이 남아 있는 경우가 많기 때문에, `strings`는 여전히 매우 유용한 1차 분석 도구다.

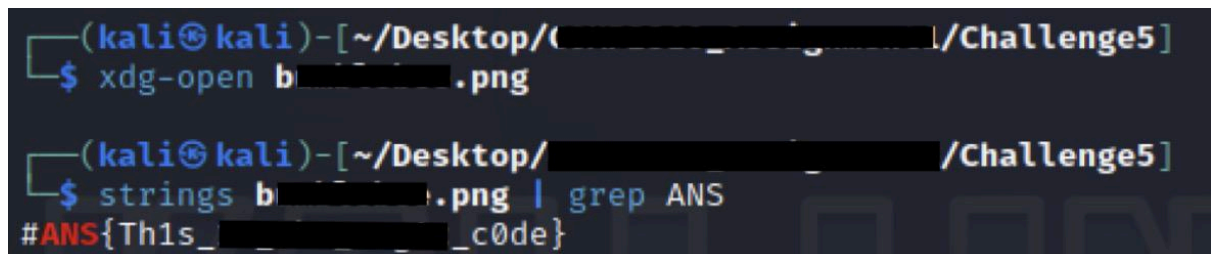


Figure 11. `strings` 명령어를 사용하여 PNG 파일 내부의 평문 문자열을 추출하고 플래그 패턴을 식별한 결과

7. 적용 기술

본 프로젝트를 통해 다음과 같은 기술적 분석 역량을 실제 사례 기반으로 적용하였다.

- 파일 시그니처 기반 분석
- 실행 파일 내부 삽입 파일 탐지
- 수동 바이너리 카빙
- CLI 기반 포렌식 분석
- APK 정적 분석 및 리소스 탐색
- Base64 인코딩 데이터 식별 및 디코딩
- 손상 파일 헤더 복구
- EXIF/XMP 메타데이터 분석
- 이미지 바이너리 내부 문자열 탐지
- 재현 가능한 분석 절차 문서화

중요한 점은, 이 기술들이 각각 독립적으로만 사용된 것이 아니라 파일 유형과 분석 상황에 따라 적절히 조합되었다는 것이다. 어떤 파일은 단순 패턴 검색으로 충분했지만, 어떤 파일은 구조 분석 후 카빙이 필요했고, 또 다른 파일은 앱 정적 분석이나 메타데이터 검사가 필요했다. 이는 단순 도구 사용 경험을 넘어, 문제 유형에 따라 분석 전략을 조정할 수 있는 능력을 보여준다.

8. 실무 적용 가능성

본 프로젝트는 교육용 CTF 환경에서 수행되었지만, 사용된 분석 논리와 기법은 실제 보안 실무와 직접적으로 연결된다. 실행 파일 내부에 이미지나 다른 데이터를 삽입하는 방식은 악성코드가 페이로드를 숨기거나 설정 정보를 은닉할 때 자주 사용하는 방식이다. 따라서 Challenge 2의 수동 카빙 경험은 악성코드 분석이나 의심 파일 구조 조사와 연결된다.

헤더 변조와 메타데이터 은닉은 파일 포맷 위장 및 정보 은닉과 직결된다. Challenge 4는 손상 파일 복구와 메타데이터 분석을 함께 수행한 사례로, 실제로는 이메일 첨부파일, 다운로드 파일, 사용자 문서, 이미지 아티팩트 조사와 유사한 맥락에서 활용될 수 있다.

또한 APK 정적 분석과 Base64 디코딩은 모바일 앱 보안과도 관련이 있다. 모바일 애플리케이션 내부에 남은 문자열, 리소스, 설정값, 토큰, 디버그 정보는 실제 보안 이슈로 이어질 수 있으며, 앱 위변조나 악성 앱 분석에서도 유사한 절차가 적용된다.

마지막으로 strings, grep, file, hexdump와 같은 기본 도구를 사용해 빠르게 이상 징후를 식별하는 능력은 Incident Response와 Threat Hunting 초기 triage에서 매우 유용하다. 모든 파일에 고급 도구를 바로 적용하기보다, 가볍고 빠른 1차 분석으로 범위를 줄이는 능력은 실무에서 오히려 큰 강점이 된다.

9. 문제 해결 과정 및 학습 내용

이번 프로젝트를 통해 가장 크게 느낀 점은, 처음부터 복잡한 기법을 적용하는 것이 항상 좋은 접근은 아니라는 것이다. 초기에는 무작위로 여러 도구를 적용하는 방식에 가까웠지만, 실제로는 그 방식이 비효율적이었다. 문자열 검색으로 충분히 해결 가능한 항목에 불필요하게 복잡한 접근을 하면 시간만 낭비하게 되고, 반대로 문자열이 보이지 않는다고 바로 포기하면 구조적 삽입 파일이나 메타데이터 은닉을 놓칠 수 있다.

따라서 이번 프로젝트를 통해 무작위 탐색보다 단계적 가설 기반 접근이 훨씬 효과적이라는 점을 체감했다. 먼저 파일 유형을 파악하고, 그 파일에 가장 적합한 기본 검사를 수행한 뒤, 결과에 따라 다음 단계로 넘어가는 방식이 전체적으로 더 안정적이었다.

또한 도구를 많이 아는 것보다, 왜 그 도구를 사용하는지 설명할 수 있는 것이 더 중요하다는 점도 배웠다. 예를 들어 **dd**를 실행하는 것 자체보다, 왜 특정 오프셋부터 잘라야 하는지 설명할 수 있어야 진짜 분석이 된다. **exiftool** 역시 단순 실행보다, 왜 시각적으로 보이지 않는 파일에서 메타데이터를 확인해야 하는지 연결할 수 있어야 의미가 있다.

마지막으로 재현 가능한 조사 절차를 기록하는 것의 중요성을 확인하였다. 결과만 적으면 나중에 동일 분석을 다시 수행하기 어렵지만, 명령어, 판단 근거, 도구, 발견 위치를 함께 기록하면 다른 분석가도 결과를 검증할 수 있다. 포렌식 보고서는 단순 결과표가 아니라, 증거와 해석을 연결하는 문서라는 점을 다시 확인한 프로젝트였다.

10. 프로젝트 회고

본 프로젝트를 통해 단순히 명령어를 실행하는 수준이 아니라, 구조적 분석 사고를 기반으로 파일을 해석하는 방식을 훈련할 수 있었다. 처음에는 **CTF**라는 형식 때문에 문제 풀이처럼 접근하기 쉬웠지만, 실제로는 각 파일이 왜 의심스러운지, 어떤 방식으로 은닉되었을 가능성이 높은지 스스로 판단해야 했기 때문에 훨씬 더 포렌식다운 사고가 필요했다.

특히 세 가지 점을 분명히 체감했다. 첫째, 파일은 겉으로 보이는 모습만으로 판단해서는 안 된다는 점이다. 열리지 않는 파일도 복구 대상이 될 수 있고, 정상적으로 열리는 파일도 내부 메타데이터나 문자열 영역에 중요한 정보를 숨기고 있을 수 있다. 둘째, 구조 이해가 곧 분석 정확도로 이어진다는 점이다. 확장자보다 시그니처를 보고, 문자열이 보이지 않으면 바이너리 구조를 확인하며, 앱 파일이라면 리소스 구조를 이해해야 한다. 셋째, 포렌식 분석은 답을 찾는 과정이 아니라 근거를 쌓는 과정이라는 점이다. 내가 왜 해당 파일을 의심했는지, 왜 이 도구를 썼는지, 결과가 어떤 해석으로 이어지는지 설명할 수 있어야 한다.

결과적으로 이 프로젝트는 **CLI** 기반 분석에 대한 자신감을 높여주었고, 동시에 무작정 도구를 사용하는 것이 아니라 근거 중심으로 조사 흐름을 설계하는 습관을 형성하는 데 큰 도움이 되었다. 포트폴리오 관점에서도, 이 프로젝트는 “도구를 사용해봤다”를 넘어 “파일 구조와 은닉 방식의 차이를 이해하고, 그에 맞는 분석 전략을 선택할 수 있다”는 점을 보여주는 사례라고 생각한다.